PROPERTY BASED TESTING

CIS 1200

TRADITIONAL UNIT TESTS

Validity is based on individual inputs and outputs

Developer's responsibility to come up with cases

Have to write many tests to cover reasonable scope

```
let test () : bool =
 (f 1) > 0
;; run_test "abs_value positive" test
```

```
let test () : bool =
 (f 5000) > 0
;; run_test "abs_value zero" test
```

```
let test () : bool =
 (f -23) > 0
;; run_test "abs_value negative" test
```

PROPERTY-BASED TESTS

Validity is based on properties of inputs and outputs

Developer describes desired property of output

One test generates many different random tests

```
let property (n : int) : bool = f n > 0
;; run_property_test “all values f(x) positive” gen_int property
```

PROPERTY-BASED TESTS

Generators: Random generators that produce values of a given type.

Properties: Claims about program behavior that should hold for many inputs.

Shrinkers: A construct that simplifies failing cases to be more interpretable.

```
let property (n : int) : bool = f n > 0
;; run_property_test “all values f(x) positive” gen_int property
```

PROPERTY-BASED TESTS

Function we've given you that will create and run the property-based test

Three parameters:

1. **Name of the Test**
2. **Generator**
3. **Property**

```
let property (n : int) : bool = f n > 0
;; run_property_test "all values f(x) positive" gen_int property
```

PROPERTY-BASED TESTS

The name of the test

```
let property (n : int) : bool = f n > 0
;; run_property_test “all values f(x) positive” gen_int property
```

PROPERTY-BASED TESTS

Property we are testing

Written as a function that takes in the generated input and returns a boolean

True = passing test

```
let property (n : int) : bool = f n > 0
;; run_property_test “all values f(x) positive” gen_int property
```

PROPERTY-BASED TESTS

Function that determines the type of the input generated

```
let property (n : int) : bool = f n > 0
;; run_property_test “all values f(x) positive” gen_int property
```

For homework 3, gen_int, gen_tree, gen_set, gen_pair, gen_triple.

Ex: gen_pair ((gen_tree gen_int) (gen_int)) generates a int tree * int!

PRACTICE

We have the function `sort_ages` which sorts `(string * int)` lists

Input:

```
lst = [(“Ryder”, 22); (“Jayme”, 20); (“Riya”, 20); (“Mehak”, 20); (“Stephen”, 21)]
```

Possible Outputs:

```
[(“Jayme”, 20); (“Riya”, 20); (“Mehak”, 20); (“Stephen”, 21); (“Ryder”, 22)]

[(“Jayme”, 20); (“Mehak”, 20); (“Riya”, 20); (“Stephen”, 21); (“Ryder”, 22)]

[(“Riya”, 20); (“Mehak”, 20); (“Jayme”, 20); (“Stephen”, 21); (“Ryder”, 22)]

. . .
```

PRACTICE

We have the function `sort_ages` which sorts `(string * int)` lists

Why might unit testing be insufficient in testing this particular function?

PRACTICE

We have the function `sort_ages` which sorts `(string * int)` lists

Why might unit testing be insufficient in testing this particular function?

If our implementation does not sort tuples of the same age in any particular order, we will have trouble testing the function on input with same-age, different-name tuples

PRACTICE

We have the function `sort_ages` which sorts `(string * int)` lists

Come up with some possible properties for testing this function!

PRACTICE

We have the function `sort_ages` which sorts `(string * int)` lists

Come up with some possible properties for testing this function!

Same-size: the sizes of the input and output must be the same.

Same-elements: the values of the input and output must be the same.

Ordered: the elements of the output (specifically, the integers) must be in ascending order.

PRACTICE

We have the function `sort_ages` which sorts `(string * int)` lists

Is this a valid property to test with? Why or why not?

```
sort_ages (sort_ages lst) = sort_ages lst
```

PRACTICE

We have the function `sort_ages` which sorts `(string * int)` lists

Is this a valid property to test with? Why or why not?

```
sort_ages (sort_ages lst) = sort_ages lst
```

No, similar to the issue with unit tests, we can't guarantee that sort_ages doesn't change even a correctly sorted output to a different correctly sorted output. Even though this looks like a convincing round-trip property, be careful when writing them!